

Athenaeum on Oracle Cloud — Part 2

Host setup, deployment, and recovery — Ubuntu 26.04 LTS

Operator edition — 8 September 2026

Continue here after [Part 1: Oracle GUI provisioning](#). Your VM is running and SSH works. This guide takes you from that checkpoint to a mounted data disk, a public website, verified backups, and automatic app updates.

Your actual OS: You installed Ubuntu **26.04**, although Part 1 specified 24.04. Use **26.04 / resolute / arm64** below. Do not rebuild the VM just to match the older guide.

How to follow this guide

Every step says where to work:

Label	Where
VM — Bash	Your SSH terminal, logged in as ubuntu
Your computer — PowerShell	A separate terminal on Windows, outside SSH
Browser	Oracle, GitHub, Docker Hub, or Cloudflare

The PowerShell examples follow Part 1. On a Linux computer, use its normal terminal; ssh and scp work there too. Linux-specific key commands appear in step 5.

Replace every `REPLACE_...` value before running a command. Run one code block at a time. If its checkpoint fails, fix that step before continuing. To edit a file with nano, save with **Ctrl+O**, **Enter**, then exit with **Ctrl+X**.

What the code actually does now

This guide was checked against the simplified Compose file, installer, host commands, CI workflows, and backup code.

- Three containers: Caddy, Athenaeum, and Quacktuaries.
- Two settings files: `compose.env` and `recovery.json`.
- The installer generates the app's session key and installs the Oracle SDK. You do not separately install OCI CLI or run `oci setup config`.
- GitHub publishes Docker Hub images with the metadata the updater requires. There are no GitHub release assets or updates.json.
- Automatic image checks run every **15 minutes**; backups run **hourly**. Neither timer starts until you enable it.
- First deployment requires empty app data. Moving the existing Cloud Run database is a separate task.

Reality-check result: Local host tests pass with Python 3.14. The live ARM/IAM checks below are still required; detailed validation notes appear at the end.

Phase A — Prepare the Linux host

1. Confirm the machine and the Part 1 handoff

VM — Bash:

```
cat /etc/os-release
uname -m
python3 --version
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS,SERIAL
findmnt /
```

Expected: Ubuntu 26.04, codename *resolute*, architecture *aarch64*, and Python 3.14.x. The patch version can differ from local tests.

Have these Part 1 values in your private notes:

- Reserved public IPv4 and SSH-key location.
- *athenaeum-data* volume size and attachment/device information.
- Data volume status **Attached**, **Paravirtualized**, **Read/write**.
- Oracle region, Object Storage namespace, and bucket *athenaeum-backups*.
- Dynamic group and both bucket/object IAM statements from Part 1.

If you created only the VM, finish Part 1's disk, bucket, and IAM steps first. Do not create duplicates.

Checkpoint: You know which disk contains Ubuntu and which is the separate data disk.

2. Identify and format only the empty data disk

Browser — Oracle: Open **Compute / Instances / athenaeum-arm / Storage / Attached block volumes**. Compare *athenaeum-data* with the Linux disk listing: size and attachment information must agree.

VM — Bash: Set the device to the verified data disk. The value below is deliberately not a real device:

```
ATHENAEUM_DEVICE=/dev/REPLACE_VERIFIED_DATA_DISK
lsblk -o NAME,PATH,SIZE,TYPE,FSTYPE,MOUNTPOINTS "$ATHENAEUM_DEVICE"
sudo wipefs --no-act "$ATHENAEUM_DEVICE"
sudo blkid "$ATHENAEUM_DEVICE"
```

Expected for a new blank disk: no filesystem signature, no partitions, and no mountpoint. *blkid* may return no output and a nonzero exit status for a blank disk.

Disk checkpoint: Never choose the disk containing */*, */boot*, or */boot/efi*. Do not assume */dev/sdb* is the data disk. If this disk already has a filesystem or records, skip formatting and identify what it contains; an existing *ext4* data disk should be mounted, not reformatted.

Only after that checkpoint, format the new blank disk:

```
sudo mkfs.ext4 -L athenaeum-data "$ATHENAEUM_DEVICE"
sudo blkid "$ATHENAEUM_DEVICE"
```

Copy the value beside `UUID=` into your private notes. This is the **filesystem UUID**, not the volume OCID or `PARTUUID`.

3. Mount the disk and prove it survives reboot

VM — Bash:

```
sudo mkdir -p /srv/athenaeum
sudo cp --no-clobber /etc/fstab /etc/fstab.before-athenaeum
sudo nano /etc/fstab
```

Add this line once, substituting the filesystem UUID:

```
UUID=REPLACE_UUID /srv/athenaeum ext4 defaults,_netdev,nofail 0 2
```

Keep all existing boot/system entries. If `/srv/athenaeum` already has an entry, review that entry instead of adding a second one.

```
sudo systemctl daemon-reload
sudo mount /srv/athenaeum
findmnt --mountpoint /srv/athenaeum -o TARGET,SOURCE,FSTYPE,UUID,OPTIONS
df -h /srv/athenaeum
```

Expected: exact mountpoint `/srv/athenaeum`, the recorded UUID, `ext4`, and `rw` among the options.

```
sudo reboot
```

The SSH session closes. Reconnect from **your computer** using the SSH command from Part 1, then rerun:

```
findmnt --mountpoint /srv/athenaeum -o TARGET,SOURCE,FSTYPE,UUID,OPTIONS
```

Checkpoint: The same filesystem is mounted after reboot. If it is missing, fix the attachment/fstab entry before installing the stack. Oracle documents UUID mounting with `_netdev,nofail`; the installer later adds Docker's stricter missing-disk guard. [Oracle mounting guidance](#).

4. Update Ubuntu and install Docker

VM — Bash: These commands are for the new Ubuntu 26.04 ARM host.

```
sudo apt update
sudo apt upgrade
sudo apt install -y ca-certificates curl git python3-venv iptables nano
```

If Ubuntu requests a reboot, reboot, reconnect, and confirm the data mount again.

Add Docker's official repository:

```

sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<'DOCKER_APT'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: resolute
Components: stable
Architectures: arm64
Signed-By: /etc/apt/keyrings/docker.asc
DOCKER_APT
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker

```

Do not use `noble` from the old guide. If Docker was already installed from another repository, resolve its conflicting packages using [Docker's Ubuntu instructions](#) before continuing.

Verify:

```

sudo docker run --rm hello-world
sudo docker compose version
sudo docker buildx version
sudo docker info --format '{{.OSType}} / {{.Architecture}}'
sudo iptables -S DOCKER-USER

```

Expected: the Docker greeting, working Compose/Buildx commands, Linux with `aarch64` or `arm64`, and a `DOCKER-USER` chain.

Keep Docker's default **iptables** firewall backend. Athenaeum's installer rejects Docker's separate `nftables` backend, `iptables=false`, or `live-restore=true`. Ubuntu's `iptables-nft` command implementation is fine; it is not the same setting as Docker's `nftables` backend. Do not flush Oracle's firewall or add a second UFW ruleset to solve Docker connectivity. [Docker firewall behavior](#).

Checkpoint: Docker works. No Athenaeum containers are running yet. Use `sudo docker`; Docker-group membership is unnecessary.

Phase B — Prepare keys and published images

5. Create the backup decryption key on your computer

This is a new **age encryption key**, separate from the SSH key. The VM needs its public half for backups. Keep the private half on your computer and in your password manager.

Your computer — PowerShell:

```
winget install --id FiloSottile.age --exact
```

Reopen PowerShell if age-keygen is not found. Then:

```
age-keygen -o "$HOME/.ssh/athenaeum-backup-identity.txt"
age-keygen -y "$HOME/.ssh/athenaeum-backup-identity.txt"
```

On a Linux computer instead: Install age through your distribution, then:

```
umask 077
age-keygen -o ~/.ssh/athenaeum-backup-identity.txt
age-keygen -y ~/.ssh/athenaeum-backup-identity.txt
```

age-keygen refuses to overwrite an existing key file. If you already created this backup key, reuse it.

Item	What to do
athenaeum-backup-identity.txt	Save the full file in your password manager; keep it out of Git
Public line beginning age1...	Copy into private notes; use it as recipient in step 10

Checkpoint: You have both the private file and its public recipient. Do not put AGE-SECRET-KEY-... in server configuration. [age installation and usage](#).

6. Protect the existing Cloud Run deployment before pushing

Your Quacktuaries repository has previously auto-deployed to Cloud Run. A push intended for Docker Hub could also replace that old container.

Browser — Google Cloud: Open the Quacktuaries project, then **Cloud Build / Triggers**. Find the trigger connected to the Quacktuaries repository. Use its row's **three-dot menu / Disable**. Check build history for an already running/queued deployment before pushing. If deployment is configured through another integration, pause that integration too. [Disable a Cloud Build trigger](#).

This does not stop the running Cloud Run service. Keep its subdomain/DNS record intact. Disabling a trigger is not a backup of its ephemeral data: preserve any required records through an agreed export/migration before

replacing that service. A game CSV alone is not a full database export.

Checkpoint: The first push for Oracle will not unexpectedly redeploy the existing Cloud Run app.

7. Put the current source in the two GitHub repositories

Your computer — IDE/Git client: Publish the Athenaeum checkout as a **public** GitHub repository named `athenaeum`. Keep Quacktuaries in its existing separate repository.

For Athenaeum, include the source, Dockerfiles, `compose.yaml`, `deploy/`, `ops/`, `ci/`, and `.github/workflows/images.yaml`. For Quacktuaries, include its current integration changes and `ci/workflow` files.

Review the selected files before committing. Do not publish `.state/`, credentials, private class data, generated databases, or backup identities. Draft Markdown in a public repository is still publicly readable source.

The workflows currently trigger on **pushes to** `main`. Use that branch, or change the workflow's branch setting before relying on publication.

Workspace reality check: At this guide's revision, the local Athenaeum Git repository had no remote and its files were untracked. A public clone URL cannot be assumed to exist yet. Complete repository publication before step 9.

8. Configure Docker Hub and run the first image builds

Browser — Docker Hub: Under your personal Docker Hub namespace, create these **public** repositories:

Repository	Built by
<code>athenaeum</code>	Athenaeum's GitHub workflow
<code>athenaeum-edge</code>	Athenaeum's GitHub workflow
<code>quacktuaries</code>	Quacktuaries' GitHub workflow

Open **Account settings / Personal access tokens / Generate new token**. Name it `athenaeum-github`; give it **Read & Write** access. Save it in your password manager. It is the CI push credential, not an Oracle credential. [Docker personal tokens](#).

Browser — each GitHub repository: Open **Settings / Secrets and variables / Actions**.

On **Variables**, click **New repository variable** for each of:

Name	Value
<code>DOCKERHUB_USERNAME</code>	Your actual Docker Hub username, not your email or GitHub username
<code>ENABLE_PUBLICATION</code>	<code>true</code>

On **Secrets**, click **New repository secret**:

Name	Value
DOCKERHUB_TOKEN	The Docker Hub token just created

Repeat in both repositories. [GitHub variables](#), [GitHub secrets](#).

Your computer — IDE: Push the reviewed changes to `main` after those settings exist. If you already pushed before configuring them, rerun **all jobs** of that push's workflow. Do not rerun only one platform job: the workflow uses attempt-specific image tags.

Browser — GitHub / Actions / Build and publish images: Wait for both check jobs and the final publish job to succeed in both repositories. Each architecture is built and smoke-tested natively. The CI runners can remain Ubuntu 24.04; that does not change your VM's OS or require editing the app Dockerfiles.

Checkpoint: All three Docker Hub repositories have a `latest` tag containing `linux/amd64` and `linux/arm64`. These must come from the current workflow: an arbitrary `docker push ...:latest` lacks the updater's required image metadata.

Phase C — Install Athenaeum on the VM

9. Clone Athenaeum into its expected location

VM — Bash: Replace the GitHub owner below. It can differ from your Docker Hub username.

```
sudo install -d -o ubuntu -g ubuntu /opt/athenaeum
git clone https://github.com/REPLACE_GITHUB_OWNER/athenaeum.git \
/opt/athenaeum/stack
cd /opt/athenaeum/stack
git status --short
ls compose.yaml ops/install-host ci/images.json
```

Expected: clean checkout and all three files present. If `/opt/athenaeum/stack` already exists, inspect it rather than deleting it or cloning over it.

Only Athenaeum's checkout is needed on the VM. Quacktuaries is pulled as an image; do not use `compose.local.yaml` or `ops/local-stack` on the production host.

10. Fill in the two configuration files

VM — Bash, in `/opt/athenaeum/stack`:

```
sudo install -d -m 0700 /etc/athenaeum
sudo cp --no-clobber deploy/production.env.example \
/etc/athenaeum/compose.env
sudo cp --no-clobber deploy/recovery.example.json \
/etc/athenaeum/recovery.json
sudo chmod 0600 /etc/athenaeum/compose.env /etc/athenaeum/recovery.json
sudo nano /etc/athenaeum/compose.env
```

Set these values:

```
DOCKERHUB_USERNAME=REPLACE_DOCKERHUB_USERNAME
SITE_DOMAIN=valentemath.com
ACME_EMAIL=REPLACE_YOUR_EMAIL
```

Use the same Docker Hub username as step 8. Do not put the push token in this file. Public image pulls do not need it.

Retrieve the exact filesystem UUID again:

```
findmnt --noheadings --output UUID --mountpoint /srv/athenaeum
sudo nano /etc/athenaeum/recovery.json
```

Fill in the existing JSON:

```
{
  "filesystem_uuid": "REPLACE_FILESYSTEM_UUID",
  "recipient": "REPLACE_PUBLIC_AGE_RECIPIENT",
  "store": {
    "region": "REPLACE_ORACLE_REGION",
    "namespace": "REPLACE_OBJECT_STORAGE_NAMESPACE",
    "bucket": "athenaeum-backups"
  }
}
```

Field	Copy from
filesystem_uuid	The findmnt command above
recipient	Public age1... line from step 5
region	Part 1's Oracle region identifier, such as us-ashburn-1
namespace	Part 1's Object Storage namespace; not the tenancy name or compartment OCID
bucket	The existing athenaeum-backups bucket

The default backup cap is **8,000,000,000 bytes**. If your remaining Object Storage allowance is smaller, add a top-level `bucket_cap_bytes` with a smaller byte count. Do not increase it based on the live disk's size: disk space and object-backup allowance are separate.

Checkpoint: Two private files, eight filled-in values, no `REPLACE_...` placeholders, and no private age identity on the VM yet.

11. Preview and apply the installer

VM — Bash:

```
cd /opt/athenaeum/stack
sudo docker compose --env-file /etc/athenaeum/compose.env \
  -f compose.yaml config --quiet
sudo ./ops/install-host --config /etc/athenaeum/recovery.json
```

Expected: Compose validation prints nothing. The installer prints a preview with `apply: false` and `services_started: false`.

Check that the preview uses your recorded UUID and `/srv/athenaeum`. Then:

```
sudo ./ops/install-host --config /etc/athenaeum/recovery.json --apply
```

The installer:

- Creates the session key for a new empty installation, preserving existing keys.
- Creates app/Caddy data directories with the required ownership.
- Installs its isolated Python environment, pinned Oracle SDK, and age tools.
- Installs `athenaeumctl`, service/timer definitions, and Docker's UUID guard.

- Saves replaced installation files under `/var/lib/athenaeum/install-rollback-....`

It does **not** mount/format disks, create cloud resources, start apps, or enable timers.

Verify the installed Python environment:

```
sudo /opt/athenaeum/operations/venv/bin/python --version
sudo /opt/athenaeum/operations/venv/bin/python \
  -c 'import oci; print(oci.__version__)'
```

Expected: Python 3.14.x on your fresh Ubuntu 26.04 host and SDK 2.185.1. If installation/import fails, keep the VM as-is and diagnose the error before starting apps; do not substitute unpinned SDK packages or reinstall Ubuntu as a first response.

12. Activate the guards and test bucket access

VM — Bash:

```
sudo systemctl enable --now athenaeum-metadata-guard.service
sudo systemctl cat docker
```

The Docker configuration must include the installed `athenaeum-recovery.conf` drop-in with:

```
[Unit]
RequiresMountsFor=/srv/athenaeum
BindsTo=srcv-athenaeum.mount
After=srcv-athenaeum.mount

[Service]
ExecStartPre=/usr/local/sbin/athenaeumctl guard-mount
```

There is no need to hand-create the old guide's second mount-guard drop-in. Activate the installed guard while this new host has no live app traffic:

```
sudo systemctl restart docker
sudo systemctl is-active docker athenaeum-metadata-guard.service
sudo athenaeumctl guard-mount
sudo athenaeumctl preflight
sudo iptables -C DOCKER-USER -d 169.254.169.254/32 \
  -m comment --comment athenaeum-block-imds -j REJECT
sudo athenaeumctl list
```

Expected: both units are active; mount verification is true; preflight reports free space; the firewall check prints nothing; list reports a JSON object containing snapshots (normally `[]` for a new bucket).

`preflight` checks local storage/configuration. `list` actually contacts Oracle using the VM's instance identity, checks the bucket settings, and lists recovery points. No OCI CLI configuration or S3 credentials are involved. [Oracle instance principals](#).

If `list` fails, check the exact region/namespace, **Private / Standard / Versioning Disabled** bucket settings, both IAM statements, and dynamic-group propagation from Part 1. This is a read-access check; step 16 verifies upload/download too.

Checkpoint: The correct disk is protected, the metadata rule exists, and the host can read the backup bucket. Do not test mount-loss by detaching a live disk.

Phase D — Bring up the website

13. Check image publication before changing DNS

VM — Bash: Substitute your Docker Hub username:

```
ATHENAEUM_HUB=REPLACE_DOCKERHUB_USERNAME
sudo docker buildx imagetools inspect \
  "docker.io/$ATHENAEUM_HUB/athenaeum:latest"
sudo docker buildx imagetools inspect \
  "docker.io/$ATHENAEUM_HUB/athenaeum-edge:latest"
sudo docker buildx imagetools inspect \
  "docker.io/$ATHENAEUM_HUB/quacktuaries:latest"
```

Expected: all three images exist, both Linux architectures appear, and index annotations include `io.valentemath.schema`, `io.valentemath.style`, and the `org.opencontainers.image.*` version/source fields. If missing, return to the GitHub Actions run; do not work around it with a different image.

14. Point the apex and www records at Oracle

Browser — Cloudflare: Open **valentemath.com / DNS / Records**. Export the existing records first so you can reverse these edits.

Set:

Type	Name	Content	Proxy
A	@	Your reserved Oracle IPv4	DNS only — gray cloud
CNAME	www	valentemath.com	DNS only — gray cloud

Use **Auto** TTL. Replace conflicts only at @ and www. Remove stale AAAA records at those names for this IPv4-only setup. Preserve email records and `quacktuaries.valentemath.com`. No Cloudflare API token is needed. [Cloudflare DNS editing](#).

Your computer — PowerShell:

```
Resolve-DnsName valentemath.com -Type A
Resolve-DnsName www.valentemath.com -Type A
```

On Linux, use `getent ahostsv4 valentemath.com` and `getent ahostsv4 www.valentemath.com`. Repeat from the VM if you suspect DNS caching there.

Checkpoint: Both names resolve to the reserved IPv4. Part 1's OCI security list allows inbound TCP **80 and 443**. Caddy needs working DNS and those ports to obtain certificates during first startup. [Caddy automatic HTTPS](#).

15. Start the new stack and exercise it

VM — Bash: This starts a fresh Oracle database; it does not import Cloud Run data.

```
sudo athenaeumctl deploy --initial
```

The command resolves approved image digests, pulls the native images, starts all three containers, and checks health and HTTPS. **Expected:** status: initialized.

```
curl -sS -o /dev/null -w '%{http_code}\n' https://valentemath.com/
curl -sS -D - -o /dev/null https://www.valentemath.com/
curl -sS -D - -o /dev/null 'https://valentemath.com/quacktuaries?source=setup'
curl -sS -o /dev/null -w '%{http_code}\n' https://valentemath.com/quacktuaries/
sudo docker ps --filter label=com.docker.compose.project=athenaeum
```

Expected: the site and app return 200; www and the bare app path return 308. The Location headers point to the apex and /quacktuaries/?source=setup respectively. All three containers are healthy. These probes use GET, matching the app's supported routes.

Confirm that the running Caddy container cannot reach Oracle's instance metadata. Copy its container ID from `docker ps` into this command:

```
sudo docker exec REPLACE_EDGE_CONTAINER_ID wget -T 3 -O /dev/null \
  --header='Authorization: Bearer Oracle' \
  http://169.254.169.254/opc/v2/instance/
```

Expected: a network refusal/timeout, not an HTTP response. The body is discarded. An HTTP response, even 401, means metadata is reachable; fix the guard before using the stack. A missing executable or wrong container ID does not count as a successful isolation test. The successful host-side `list` in step 12 verifies that host IAM access still works.

Browser:

1. Open the site and its Projects / Quacktuaries link.
2. Log in with a synthetic teacher name and create a test game.
3. Join as a synthetic student in another browser profile.
4. Start the game, perform a student action, end it, and export its CSV.

The teacher login currently uses Quacktuaries' existing name-and-cookie ownership model; this guide does not add password accounts. Use test records until live acceptance/migration is complete.

Checkpoint: The new URL works end to end. Leave all test sessions **ended** so the automatic updater will not defer them as active classes.

`sudo athenaeumctl status` can report a stale/missing backup at this point; that is expected until step 16. If the first deployment fails, do not delete its database or deployment state and rerun `--initial`: it has no prior image to roll back to. Use the troubleshooting section below.

Phase E — Prove recovery, then enable automation

16. Make the first encrypted backup

VM — Bash:

```
sudo athenaeumctl backup
sudo athenaeumctl list
```

Expected: a successful backup reports snapshot, ciphertext_sha256, and verified_at; list includes that snapshot's id.

Copy the snapshot ID and ciphertext checksum into your private notes. The command has taken a consistent SQLite snapshot, encrypted it, uploaded it, downloaded it again, and verified the stored bytes. An upload alone does not prove that you can decrypt and run it; do the next step.

17. Temporarily supply the decryption key for a restore drill

The private key is needed only for this drill, not scheduled backups.

VM — Bash: Create a private transfer directory:

```
install -d -m 0700 /home/ubuntu/.athenaeum-key-transfer
```

Your computer — PowerShell: Copy your saved private key file into it:

```
scp -i "$HOME/.ssh/athenaeum-oracle" `
"$HOME/.ssh/athenaeum-backup-identity.txt" `
ubuntu@REPLACE_VM_IP:~/.athenaeum-key-transfer/identity.txt
```

On Linux, use the same scp arguments with \ instead of PowerShell's backtick for line continuation.

VM — Bash: Move the temporary copy into root-readable runtime storage:

```
chmod 0600 /home/ubuntu/.athenaeum-key-transfer/identity.txt
sudo install -m 0600 /home/ubuntu/.athenaeum-key-transfer/identity.txt \
/run/athenaeum-restore-identity
rm /home/ubuntu/.athenaeum-key-transfer/identity.txt
rmdir /home/ubuntu/.athenaeum-key-transfer
```

Your original key on your computer remains unchanged. The VM copy under /run will be removed after the drill.

18. Restore into a separate directory and run the recovered app

VM — Bash: Find the running Quacktuaries image reference:

```
sudo docker ps \
  --filter label=com.docker.compose.project=athenaeum \
  --filter label=com.docker.compose.service=quacktuaries \
  --format '{{.Image}}'
```

Copy the full `docker.io/.../quacktuaries@sha256:... reference`. Updates are still disabled, so it should be the image recorded in your first backup.

Set these two shell variables using actual values:

```
ATHENAEUM_SNAPSHOT='REPLACE_SNAPSHOT_ID'
ATHENAEUM_IMAGE='REPLACE_FULL_QUACKTUARIES_IMAGE_REFERENCE'
sudo athenaeumctl restore-test \
  --snapshot "$ATHENAEUM_SNAPSHOT" \
  --target /var/tmp/athenaeum-restore-first \
  --identity /run/athenaeum-restore-identity \
  --image "$ATHENAEUM_IMAGE"
```

The target directory must **not already exist**. On a later drill, choose a new name. Never point it at `/srv/athenaeum`.

Expected: runtime: passed and network: none. The test validates the archive/database and starts a separate copy of the recovered app in a temporary container with no network or published ports. It removes that container; the private restored directory remains for inspection. It does not overwrite live records.

After the test finishes, whether it passes or fails, remove the temporary VM key:

```
sudo rm /run/athenaeum-restore-identity
```

Checkpoint: The Oracle backup decrypted and the restored app ran on your ARM host. If it fails, resolve it before enabling automatic updates.

19. Download an encrypted copy outside Oracle

VM — Bash: Use the same snapshot ID from step 18:

```
sudo athenaeumctl export --snapshot "$ATHENAEUM_SNAPSHOT" \
  --output /home/ubuntu/athenaeum-first-backup.age
sudo chown ubuntu:ubuntu /home/ubuntu/athenaeum-first-backup.age
chmod 0600 /home/ubuntu/athenaeum-first-backup.age
```

Save the reported `ciphertext_sha256`. The export refuses an existing output file; choose a new filename for future copies.

Your computer — PowerShell:

```
scp -i "$HOME/.ssh/athenaeum-oracle" `
  ubuntu@REPLACE_VM_IP:athenaeum-first-backup.age `
  "$HOME/Downloads/athenaeum-first-backup.age"
Get-FileHash "$HOME/Downloads/athenaeum-first-backup.age" -Algorithm SHA256
```

On Linux, use `scp` and `sha256sum` instead. Compare the checksum with the VM's value; letter case does not matter. Keep the encrypted archive and recovery notes outside Oracle, with the private identity saved independently.

20. Enable hourly backups and automatic updates

VM — Bash, only after the backup and restore checkpoints passed:

```
sudo systemctl enable --now athenaeum-backup.timer
sudo systemctl enable --now athenaeum-updates.timer
sudo systemctl list-timers 'athenaeum-*'
sudo athenaeumctl status
```

Expected: both timers are listed with future runs; the data disk has free space; the backup is recent; all three containers are running/healthy. Updates: allowed means not paused; check the timer lines to confirm scheduling is actually active.

Then reboot once during this setup window:

```
sudo reboot
```

Reconnect and verify:

```
findmnt --mountpoint /srv/athenaeum -o TARGET,FSTYPE,UUID
sudo systemctl is-active docker athenaeum-metadata-guard.service
sudo systemctl list-timers 'athenaeum-*'
sudo athenaeumctl status
```

Open the website and confirm your ended test game's records remain. If you kept the browser cookies, check teacher/student access too.

Checkpoint: Data, services, guards, and timers survive reboot.

21. Use the normal maintenance commands

Edit Markdown or app code in your IDE, commit, and push to GitHub. After CI succeeds, the next eligible 15-minute check updates the changed site/app. Quacktuaries waits for classes/lobbies to end and takes a verified backup first. Caddy and infrastructure changes remain deliberate operations.

Command on the VM	Purpose
<code>sudo athenaeumctl status</code>	Readable health/backup/update summary
<code>sudo athenaeumctl status --json</code>	Full diagnostic details and image records
<code>sudo athenaeumctl deploy</code>	Check for eligible app updates now
<code>sudo athenaeumctl backup</code>	Make a backup now

Command on the VM	Purpose
<code>sudo athenaeumctl pause-updates</code>	Prevent subsequent automatic updates, including after reboot
<code>sudo athenaeumctl resume-updates</code>	Allow subsequent update checks
<code>sudo athenaeumctl rollback quacktuaries</code>	Restore the previous compatible image without replacing data

A failed image stays held until a corrected image is published. Schema-changing releases need a maintenance plan; they are not ordinary unattended updates. A pause does not interrupt an operation already holding the lock. See [delivery](#) for scheduling and rollback details, and [recovery](#) for later restore drills.

Monthly: check disk space, backup age, Oracle usage/account notices, and Ubuntu updates. Download a current encrypted backup and repeat the restore drill. Do not use volume pruning as routine maintenance.

Troubleshooting and final handoff

If a checkpoint fails

Symptom	First action
Data disk missing after reboot	Recheck the existing attachment and fstab UUID. Do not reformat.
Docker apt repository error	Verify /etc/os-release; this VM uses resolute, not noble.
Installer rejects UID or session key	Inspect the named existing file/directory. Do not recursively chown a data disk or generate a new key over existing records.
DOCKER-USER missing	Check Docker is active and using its iptables backend.
list says unauthorized/not found	Check region, namespace, bucket settings, instance-specific dynamic-group rule, both policies, and propagation.
preflight passes but backups fail	Preflight is local. Check bucket permissions, quota/headroom, and backup-service logs.
Image/index/annotation error	Confirm both repositories' current CI runs finished publishing all images.
HTTPS fails	Check apex/www A/AAAA/CAA records, DNS-only mode, OCI 80/443 rules, and Caddy logs.
Updates are deferred	End abandoned test lobbies/classes; inspect pause, retry backoff, and timer state.
Backup cap reached	Preserve the last good backup; review usage/retention and available allowance before deleting objects or raising the cap.
Restore target already exists	Choose a new isolated target name. Do not overwrite live data.

Useful **VM** diagnostics:

```
sudo athenaeumctl status --json
sudo journalctl -u docker -n 50 --no-pager
sudo journalctl -u athenaeum-backup.service -n 50 --no-pager
sudo journalctl -u athenaeum-updates.service -n 50 --no-pager
sudo docker ps -a --filter label=com.docker.compose.project=athenaeum
```

To see a container's logs, copy its ID from the last command and run `sudo docker logs --tail 50 CONTAINER_ID`.

Failed first deployment: `deploy --initial` may have created data and recorded an initial transaction before a certificate/readiness failure. Preserve both. Fix the reported cause, then review the stored state with an agent before trying to initialize again. There is no automatic reset command or previous image for this first-start case.

Keep logs private when they contain URLs, records, or configuration. No tokens or private keys need to be pasted into a support request.

Final checklist

- [] Ubuntu 26.04 ARM host and exact data UUID confirmed.
- [] Docker works and the installed disk/metadata guards are active.
- [] Two private configuration files and a persistent session key exist.
- [] Both native CI jobs and image publication succeed for both repositories.
- [] `valentemath.com`, `www`, and `/quacktuaries/` behave correctly.
- [] A consistent backup was uploaded, downloaded, and verified.
- [] An isolated restore ran successfully on this ARM VM.
- [] An encrypted recovery copy exists outside Oracle.
- [] Temporary decryption-key copy was removed from the VM.
- [] Both timers and app records survived reboot.
- [] The original Cloud Run app/subdomain and its required data were preserved.

Next: choose/apply the shared visual style, then plan the existing Quacktuaries data migration and old-subdomain behavior. This guide sets up a new Oracle instance of the app; it does not authorize retiring Cloud Run or claim that migration is complete.

Validation notes

Reality-check result: 44 host tests passed under Python 3.12 and an isolated Python 3.14 runtime. The pinned Oracle SDK imported and accepted the backup API arguments without network calls; dependency resolution found ARM64/Python 3.14 wheels. This is local compatibility evidence, not a completed Ubuntu 26.04 ARM deployment. The live checks below verify your host and IAM. Oracle's published SDK support table does not yet explicitly certify this combination. [Ubuntu 26.04 changes](#), [Oracle SDK support](#).

Athenaeum / Part 2 — Host setup and deployment — Operator edition — 8 September 2026